# EECS 348 Assignment 3 – GenAI Analysis

Bikramaditya Kowsik

Student ID 3221682

October 1, 2026

Task: CEO email priority queue in C++ using objects and a from-scratch, list-based MaxHeap.

## a. GenAI programs used

**ChatGPT** (OpenAI), the free web version.

**Gemini** (Google), the free web version. I believe the model was Gemini Flash-Lite.

## b. How I accessed them

**ChatGPT:** I searched for ChatGPT on Google, clicked the result, and used chatgpt.com in my web browser. The link I used was:

```
https://chatgpt.com/?utm_source=google&utm_medium=paid_search&gbraid=0AAAAA-I0E5dHX8uJ81C46mDivm3BoWe1j&gclid=CjwKCAjwwrPVBhA1EiwAv_YO-U7nNc8Z10rW-eCm2uFYM1SyjAKdVN-w5hHgBz-YTRwNsGR5tvhPnxoCWEEQAvD_BwE&c_id=23637266097&c_crid=799723648521&c_agid=196969197751&c_kwid=kwd-1931186779120
```

- **Gemini:** I used the Gemini web app in my browser. Link:

```
https://gemini.google.com/app/95648e7880c35534
```

## c. Prompt used (identical for both GenAIs)

I pasted the exact text below into both ChatGPT and Gemini, with no edits between them.

```
Write a complete C++ program (single file, C++11, compiles with g++ on Linux) that prioritizes
emails for a busy company CEO. Use classes/objects, not standalone functions (only main() may be
outside a class). Implement a MaxHeap from scratch as the priority queue using a list-based
(dynamic array) implementation. Do NOT use std::priority_queue, std::make_heap, push_heap,
pop_heap, or any other pre-existing heap module.

Priority rules: sender categories in order of reading: Boss (first), Subordinate, Peer,
ImportantPerson, OtherPerson (last). If two emails have the same sender category, the newest
email (by date) is read first. Dates are in MM-DD-YYYY format.

The program reads a test file (filename passed as a command-line argument). Each line is one
command:
- EMAIL <sender category>,<subject line>,<date>  (EMAIL is followed by a space; the fields are
comma-delimited; the subject may contain spaces but not commas)
- NEXT: display the highest-priority email without removing it, in this format:
Next email:
Sender: <sender>
Subject: <subject>
Date: <date>
- READ: remove the highest-priority email from the queue (nothing is displayed)
- COUNT: print "There are N emails to read."

Edge cases: NEXT or READ when the queue is empty must not crash; two NEXTs in a row give
identical output; two READs in a row delete two emails without displaying them.

Sample input:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

```
Expected output:
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

# d. Generated code from both GenAIs (raw, unedited)

The code is exactly what each GenAI returned. I removed only the surrounding explanation text and markdown fences, so that only the program remains. I did not change any code.

## d.1 ChatGPT raw code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>


using namespace std;


class Email {
private:
    string sender;
    string subject;
    string date;
    int categoryPriority;


public:
    Email() : sender(""), subject(""), date(""), categoryPriority(0) {}


    Email(const string& s, const string& sub, const string& d)
        : sender(s), subject(sub), date(d) {
        categoryPriority = getCategoryPriority(s);
    }


    int getCategoryPriority(const string& s) const {
        if (s == "Boss") return 5;
        if (s == "Subordinate") return 4;
        if (s == "Peer") return 3;
        if (s == "ImportantPerson") return 2;
        return 1; // OtherPerson
    }


    string getSender() const {
        return sender;
    }


    string getSubject() const {
        return subject;
    }


    string getDate() const {
        return date;
    }


    int getPriority() const {
        return categoryPriority;
    }


    // Convert MM-DD-YYYY into YYYYMMDD so dates can be compared numerically.
    long long getDateValue() const {
        int month = 0, day = 0, year = 0;
        char dash1, dash2;


        stringstream ss(date);
        ss >> month >> dash1 >> day >> dash2 >> year;


        return static_cast<long long>(year) * 10000LL
             + static_cast<long long>(month) * 100LL
             + day;
    }


    // Returns true if this email has higher priority than other.
    bool higherPriorityThan(const Email& other) const {
        if (categoryPriority != other.categoryPriority)
            return categoryPriority > other.categoryPriority;


        return getDateValue() > other.getDateValue();
    }
};



class MaxHeap {
private:
    Email* heap;
    int size;
    int capacity;


    void resize() {
        int newCapacity = capacity * 2;
        Email* newHeap = new Email[newCapacity];


        for (int i = 0; i < size; ++i)
            newHeap[i] = heap[i];
```

```
        delete[] heap;
        heap = newHeap;
        capacity = newCapacity;
    }

    void swapEmails(Email& a, Email& b) {
        Email temp = a;
        a = b;
        b = temp;
    }

    void siftUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;

            if (!heap[index].higherPriorityThan(heap[parent]))
                break;

            swapEmails(heap[index], heap[parent]);
            index = parent;
        }
    }

    void siftDown(int index) {
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;

            if (left < size &&
                heap[left].higherPriorityThan(heap[largest])) {
                largest = left;
            }

            if (right < size &&
                heap[right].higherPriorityThan(heap[largest])) {
                largest = right;
            }

            if (largest == index)
                break;

            swapEmails(heap[index], heap[largest]);
            index = largest;
        }
    }

public:
    MaxHeap() {
        capacity = 10;
        size = 0;
        heap = new Email[capacity];
    }

    ~MaxHeap() {
        delete[] heap;
    }

    void insert(const Email& email) {
        if (size == capacity)
            resize();

        heap[size] = email;
        ++size;
        siftUp(size - 1);
    }

    bool isEmpty() const {
        return size == 0;
    }

    int getSize() const {
        return size;
    }

    const Email* getMax() const {
        if (isEmpty())
            return nullptr;

        return &heap[0];
    }

    void removeMax() {
        if (isEmpty())
            return;

        heap[0] = heap[size - 1];
        --size;

        if (size > 0)
            siftDown(0);
    }
```

```
};



class EmailManager {
private:
    MaxHeap emailQueue;


    void processEmail(const string& command) {
        // Remove "EMAIL " from the beginning.
        string data = command.substr(6);


        string sender;
        string subject;
        string date;


        size_t firstComma = data.find(',');
        size_t secondComma = data.find(',', firstComma + 1);


        if (firstComma == string::npos || secondComma == string::npos)
            return;


        sender = data.substr(0, firstComma);
        subject = data.substr(firstComma + 1,
                              secondComma - firstComma - 1);
        date = data.substr(secondComma + 1);


        emailQueue.insert(Email(sender, subject, date));
    }


    void processNext() {
        const Email* email = emailQueue.getMax();


        if (email == nullptr)
            return;


        cout << "Next email:" << endl;
        cout << "Sender: " << email->getSender() << endl;
        cout << "Subject: " << email->getSubject() << endl;
        cout << "Date: " << email->getDate() << endl;
    }


    void processRead() {
        emailQueue.removeMax();
    }


    void processCount() {
        cout << "There are " << emailQueue.getSize()
             << " emails to read." << endl;
    }


public:
    void processCommand(const string& command) {
        if (command.compare(0, 6, "EMAIL ") == 0) {
            processEmail(command);
        }
        else if (command == "NEXT") {
            processNext();
        }
        else if (command == "READ") {
            processRead();
        }
        else if (command == "COUNT") {
            processCount();
        }
    }


    void processFile(const string& filename) {
        ifstream input(filename);


        if (!input)
            return;


        string line;


        while (getline(input, line)) {
            processCommand(line);
        }
    }
};



int main(int argc, char* argv[]) {
    if (argc < 2)
        return 1;


    EmailManager manager;
    manager.processFile(argv[1]);


    return 0;
}
```

## d.2 Gemini raw code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>


class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;


    int getCategoryRank() const {
        if (sender == "Boss") return 5;
        if (sender == "Subordinate") return 4;
        if (sender == "Peer") return 3;
        if (sender == "ImportantPerson") return 2;
        if (sender == "OtherPerson") return 1;
        return 0; // Default fallback
    }


    long getDateNumeric() const {
        // Converts MM-DD-YYYY to YYYYMMDD for easy chronological comparison
        if (date.length() != 10) return 0;
        int month = std::stoi(date.substr(0, 2));
        int day = std::stoi(date.substr(3, 2));
        int year = std::stoi(date.substr(6, 4));
        return (long)year * 10000 + (long)month * 100 + (long)day;
    }


public:
    Email(const std::string& s, const std::string& sub, const std::string& d)
        : sender(s), subject(sub), date(d) {}


    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }


    bool operator<(const Email& other) const {
        int rank1 = this->getCategoryRank();
        int rank2 = other.getCategoryRank();
        if (rank1 != rank2) {
            return rank1 < rank2;
        }
        return this->getDateNumeric() < other.getDateNumeric();
    }
};


class MaxHeap {
private:
    std::vector<Email> data;


    void siftUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (data[parent] < data[index]) {
                std::swap(data[parent], data[index]);
                index = parent;
            } else {
                break;
            }
        }
    }


    void siftDown(int index) {
        int size = data.size();
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;


            if (left < size && data[largest] < data[left]) {
                largest = left;
            }
            if (right < size && data[largest] < data[right]) {
                largest = right;
            }
            if (largest != index) {
                std::swap(data[index], data[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }


public:
    void push(const Email& email) {
        data.push_back(email);
        siftUp(data.size() - 1);
    }


    void pop() {
        if (data.empty()) return;
        data[0] = data.back();
        data.pop_back();
```

```
        if (!data.empty()) {
            siftDown(0);
        }
    }

    Email top() const {
        return data[0];
    }

    bool empty() const {
        return data.empty();
    }

    size_t size() const {
        return data.size();
    }
};

class CommandProcessor {
private:
    MaxHeap heap;

public:
    void processFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error opening file: " << filename << std::endl;
            return;
        }

        std::string line;
        while (std::getline(file, line)) {
            // Trim trailing carriage returns if running on Windows files
            if (!line.empty() && line.back() == '\r') {
                line.pop_back();
            }
            if (line.empty()) continue;

            if (line.rfind("EMAIL ", 0) == 0) {
                std::string rest = line.substr(6);
                size_t firstComma = rest.find(',');
                size_t lastComma = rest.rfind(',');

                if (firstComma != std::string::npos && lastComma != std::string::npos && firstComma != lastComma) {
                    std::string sender = rest.substr(0, firstComma);
                    std::string subject = rest.substr(firstComma + 1, lastComma - firstComma - 1);
                    std::string date = rest.substr(lastComma + 1);
                    heap.push(Email(sender, subject, date));
                }
            } else if (line == "COUNT") {
                std::cout << "There are " << heap.size() << " emails to read." << std::endl;
            } else if (line == "NEXT") {
                if (!heap.empty()) {
                    Email e = heap.top();
                    std::cout << "Next email:\n"
                              << "Sender: " << e.getSender() << "\n"
                              << "Subject: " << e.getSubject() << "\n"
                              << "Date: " << e.getDate() << std::endl;
                }
            } else if (line == "READ") {
                if (!heap.empty()) {
                    heap.pop();
                }
            }
        }
    }
};

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <test_file>\n";
        return 1;
    }

    CommandProcessor processor;
    processor.processFile(argv[1]);

    return 0;
}
```

## e. Analysis of the two programs

I compiled both programs exactly as generated with `g++ -std=c++11 -O2` and tested them with the sample file from the assignment, with Windows-style (CRLF) line endings, with ties, with a malformed date, with an empty queue, and with a 300,000-email stress file. Timing figures are the best of 3 runs on a Linux machine; absolute times will differ on the Cycle servers, but the ratios are what matter.

### e.1 Correctness

| Test | ChatGPT | Gemini |
|---|---|---|
| Assignment sample file | Correct (matches the expected output exactly) | Correct (matches the expected output exactly) |
| Sample file with Windows (CRLF) line endings | **Fails: prints nothing at all.** It never strips the carriage return, so `NEXT\r`, `READ\r` and `COUNT\r` match no command. | Correct. It removes a trailing carriage return from every line. |
| Malformed 10-character date (e.g. `ab-cd-efgh`) | No crash. Its stringstream parse silently produces a meaningless date value. | **Crashes** with an uncaught `std::invalid_argument` from `std::stoi` (exit code 134). |
| Three Boss emails on the same date | Pops in an arbitrary order (First, Third, Second) | Pops in an arbitrary order (First, Third, Second) |
| NEXT/READ on an empty queue; two NEXTs in a row; two READs in a row | Correct. Prints nothing when empty, and repeated NEXTs give the same output. | Correct. Same behavior. |
| Unrecognized sender string | Silently treated as OtherPerson (rank 1) | Ranked below everyone (rank 0) |

Both programs implement the priority rules correctly: Boss, Subordinate, Peer, ImportantPerson, OtherPerson, with the newest email first inside a category. Both handle the empty-queue cases and double NEXT/READ correctly. Their real weaknesses are different: ChatGPT breaks on CRLF input, which could matter if the grader's test file was created on Windows, and Gemini can crash on a malformed date. Neither defines the order of two emails with the same category and date.

### e.2 Execution time

Both programs are O(log n) per EMAIL and per READ, and O(1) per NEXT and COUNT, so total time for n emails is O(n log n). The difference is the constant factor. In both programs the comparison function is called about 2 to 3 times per level of the heap, and each call re-derives the priority from strings: ChatGPT builds a `stringstream` for every date comparison, and Gemini re-runs `substr` and `std::stoi` three times per date plus a chain of string comparisons for the category. Both also print with `std::endl`, which flushes the output after every line.

| Measurement (300,000 emails) | ChatGPT | Gemini |
|---|---|---|
| Insert 300,000 emails then READ them all (no NEXT output) | 2.93 s | 1.06 s |
| Insert 300,000, then 300,000 NEXT+READ pairs (1.2 million output lines) | 5.49 s | 1.93 s |

Gemini was about 2.8 times faster than ChatGPT on the heap work, mainly because `substr` plus `stoi` is cheaper than building a `stringstream` on every comparison. ChatGPT is slowest overall.

### e.3 Space complexity

Both use O(n) space for n emails. ChatGPT stores the heap in a hand-managed `new[]` array that doubles when full (starting at capacity 10), so growth is amortized O(1) per insert, but it copies three Email objects for every swap and the class has a destructor with no copy constructor or copy assignment, so copying a `MaxHeap`

would cause a double `delete[]`. Gemini stores the heap in a `std::vector<Email>`, which is memory-safe and also doubles internally, but `top()` returns a full copy of an Email and `pop()` copies the last element. Measured peak memory for 300,000 emails was about 69 MB for ChatGPT and about 61 MB for Gemini.

### e.4 Maintainability

| Aspect | ChatGPT | Gemini |
|---|---|---|
| Structure | Three clear classes: Email, MaxHeap, EmailManager. Parsing is split into small private methods (processEmail, processNext, processRead, processCount). | Three clear classes: Email, MaxHeap, CommandProcessor, but the entire command parser sits in one long `processFile` method. |
| Naming and API | Descriptive names (higherPriorityThan, getMax returns a pointer that is null when empty, removeMax). Safe peek. | Overloads `operator<` for the heap. `top()` on an empty heap would be undefined behavior if called without the empty check. |
| Magic numbers | Ranks 5, 4, 3, 2, 1 are bare numbers; capacity 10 is a bare number. | Ranks 5, 4, 3, 2, 1, 0 and the date offsets are bare numbers. |
| Style problems | `using namespace std;`, no comments, no copy protection for the owning pointer. | Includes `<algorithm>` only for `std::swap`, no comments, error handling missing for bad dates. |
| Fit to the assignment | The MaxHeap is built entirely by hand on a raw dynamic array, which is exactly a list-based implementation from scratch. | Relies on `std::vector` for storage. This is not a heap module, but the heap's storage is not hand-built. |

## f. Selection and justification

**I selected the ChatGPT program as the basis for this assignment.**

My reasons, tied to the analysis above:

- **It matches the requirement most directly.** The assignment says to implement the MaxHeap from scratch with a list-based implementation. ChatGPT's heap manages its own dynamic array, handles growth and frees memory itself. Gemini delegates storage to `std::vector`.
- **Its API is safer.** `getMax()` returns a null pointer on an empty heap and nothing is copied on a peek, whereas Gemini's `top()` returns a copy and is undefined on an empty heap.
- **Its structure is easier to extend.** Each command has its own small method, so I could fix CRLF handling in one place (the command dispatcher) without touching the parser logic. Gemini's single long `processFile` mixes input handling, parsing and output.
- **Its weaknesses are localized and cheap to fix.** The measured slowness (2.93 s vs 1.06 s) comes almost entirely from one method, `getDateValue()`, which rebuilds a stringstream on every comparison. The CRLF failure is one missing trim. Both can be fixed without redesigning anything.

**Trade-off I accepted.** Gemini beat ChatGPT on raw speed (about 2.8 times faster) and on CRLF handling. I did not pick Gemini because its measured speed advantage came from a cheaper parse that I could replace with a one-time parse anyway, its crash on bad dates is also fixable, and the structural advantages above favored ChatGPT. I carried Gemini's CRLF idea into my version.

## g. Improvements I made

My final program is in `email_priority.cpp`. It is built on the ChatGPT baseline. Every line is commented, and the lines I wrote or changed are marked [My work].

### g.1 Correctness

- Added a `trim()` function and apply it to every input line, so Windows (CRLF) files and stray spaces no longer break NEXT, READ and COUNT. **Result:** the CRLF sample file went from no output at all to the exact expected output.
- Replaced the stringstream date parser with a validated digit-by-digit parser. A malformed date can never throw or crash; it becomes the lowest date value. **Result:** the 10-character malformed date that crashed Gemini now runs normally.
- Added an arrival-order sequence number as the last tie-breaker. **Result:** three same-category, same-date emails now come out in the order they arrived (First, Second, Third) instead of an arbitrary order (First, Third, Second).
- Unrecognized sender categories are ranked below all five valid ones instead of being counted as OtherPerson.
- Added an error message when the input file cannot be opened, and a standard-input fallback when no file name is given.
- Verified that the assignment's sample file still produces the exact expected output.

## g.2 Execution time

- Parse the category rank and the date **once**, in the Email constructor, and store them as integers. Each heap comparison is now two integer comparisons instead of a stringstream parse or string comparisons.
- Replaced repeated three-copy swaps in sift-up and sift-down with a "hole" technique that makes one copy of the moving email and one assignment per level.
- Replaced `std::endl` with a newline so the output stream is not flushed after every line.

| Measurement (300,000 emails) | ChatGPT | Gemini | My improved version |
|---|---|---|---|
| Insert 300,000 then READ all | 2.93 s | 1.06 s | **0.38 s** (7.7x faster than ChatGPT, 2.8x faster than Gemini) |
| Insert 300,000, then 300,000 NEXT+READ pairs | 5.49 s | 1.93 s | **0.53 s** (10.3x faster than ChatGPT, 3.6x faster than Gemini) |

The big-O stays O(log n) per insert and remove and O(1) for NEXT and COUNT, so the improvement is entirely a smaller constant factor. In my own measurements, removing `std::endl` alone cut the full run from about 2.0 s to about 0.5 s.

## g.3 Space complexity

- Space is still O(n), but I removed the memory-safety problem: I deleted the copy constructor and copy assignment of `MaxHeap`, so a copied heap can never double-free the array.
- The hole technique eliminates the temporary Email copies that every swap created.
- Stored the date as a 4-byte `int` (the largest value, 99991231, fits in an int) instead of an 8-byte `long long`.
- **An honest trade-off:** measured peak memory for 300,000 emails rose to about 114 MB, versus about 69 MB for ChatGPT and about 61 MB for Gemini. This is because each Email now caches a rank, a date value and a sequence number (more bytes per email), and because starting at capacity 16 and doubling lands on 524,288 slots for 300,000 emails, while ChatGPT's capacity of 10 lands on 327,680. I accepted this because it is a constant factor, not a change in complexity, and the speed and correctness gains were larger.

## g.4 Maintainability

- Replaced the magic numbers with a named `Rank` enumeration (RANK_BOSS, RANK_SUBORDINATE, and so on).

- Removed `using namespace std` and qualified names with `std::`.
- Made methods const-correct (for example `processNext` and `processCount`), and moved date and rank parsing into small private static helpers.
- Added a full prolog comment (program name, description, inputs, output, collaborators, other sources, author, creation and revision dates, revisions) and a comment on every line, with my own lines marked [My work].
- Used one stream-processing method for both file and standard-input reading, so the command logic exists in only one place.

## Final result

My final program passes the assignment sample file with exactly the expected output, handles CRLF input, handles empty-queue NEXT/READ, repeated NEXTs and repeated READs, never crashes on malformed input, and was measured at 7.7 to 10.3 times faster than the ChatGPT baseline and 2.8 to 3.6 times faster than the Gemini program, at the cost of higher peak memory.